

Өгөгдлийн бүтэц ICSI - 202 хичээлийг явцын шалгалтын даалгавар

*Гүйцэтгэсэн - Програм хангамж хөтөлбөрийн
2-р түвшиний оюутан - Н. Мөнхпүрэв /23B1NUM0114/*

Текст редакторлах программуудад, загвар мөрийн буюу дүрийн өгсөн текст доторх бүх тохиролыг олох шаардлага үргэлж гардаг. Ерөнхийдөө, текст нь боловсруулах гэж буй баримт бичиг, хайх дүр нь хэрэглэгчийн өгсөн тодорхой үг байна. Мөр хайлт гэж нэрлэдэг энэ бодлогыг бодох үр ашигтай алгоритмууд нь текст редакторлах программын хүлээх үүрэгт ихээхэн туслаж чаддаг. Тэдгээрийн өөр олон хэрэглээнүүдэд мөр хайлтын алгоритмууд нь ДНХ (DNA) дарааллуудаас тодорхой дүрийг хайдаг.

Naive хайлтын алгоритм

Ажиллах зарчим:

Naive тэмдэгт мөр хайлтын арга нь өгөгдсөн текст дотроос тодорхой дүрийг хайх энгийн бөгөөд шууд арга юм. Энэ алгоритм нь текстийн эхнээс эхлэн дүрийг нэг нэгээр харьцуулж, тэнцүү эсэхийг шалгадаг. Хэрэв таарахгүй бол дүрийн байрлал шилжилт хийж дахин харьцуулна. Энэ процесс текст дуустал эсвэл дүр олдтол үргэлжилнэ.

Хугацаа болон санах ойн шинжилгээ:

Хугацааны нарийвчлал (Time Complexity):

- **Хамгийн сайн тохиолдол (Best Case):** $O(m)$
 - m = загварын урт
 - Жишээ: Дүр текст дээр тэр дор нь олдвол (эхний байрлалд таарвал).
- **Хамгийн муу тохиолдол (Worst Case):** $O((n - m + 1) * m)$
 - n = текстийн урт, m = дүрийн урт
 - Жишээ: Хэрэв дүр текстэд байхгүй эсвэл сүүлд орвол, алгоритм текст дээрх бүх боломжит байрлалд $(n - m + 1)$ харьцуулна тус бүрт m тэмдэгт шалгана.
- **Дундаж тохиолдол (Average Case):** $O((n - m + 1) * m)$
 - Ихэнх практик тохиолдолд харьцуулалт текст дуустал үргэлжилнэ.

Санах ойн нарийвчлал (Space Complexity): $O(1)$

1. Зөвхөн хэдхэн хувьсагч (жишээ нь индекс) хэрэглэдэг тул нэмэлт санах ой бараг шаардлагагүй.

Давуу талууд

- **Ойлгоход хялбар байдал:** Алгоритм энгийн тул ойлгоход хялбар, хэрэгжүүлэхэд амар.
- **Урьдчилсан боловсруулалт шаардлагагүй:** Текст эсвэл дүрийг урьдчилан боловсруулах шаардлагагүй.
- **Бүх тохиолдолд ажиллана:** Ямар ч тэмдэгтийн багц эсвэл хэлний текст дээр ажилладаг.

Сул талууд

- **Хугацааны өндөр зардал:** Харьцуулалт олон удаа хийгддэг тул удаан ажилладаг.
- **Үр ашиггүй байдал:** Текст дотор давтагддаг хэсгийг анхаарч ашигладаггүй.
- **Том өгөгдөлд тохиромжгүй:** Том хэмжээний текст дээр хайлт хийхэд маш удаан болдог. Практикт хэрэглэхэд хязгаарлагдмал.

Тохиромжтой хэрэглээний тохиолдол

1. **Жижиг текст эсвэл загвар хайх:** Текст болон загвар богино бол үр ашигтай ажиллана.
2. **Прототип хөгжүүлэлт:** Хурдан бөгөөд энгийн шийдэл шаардлагатай үед.
3. **Сургалтын зорилго:** Алгоритмын үндсийг ойлгоход тохиромжтой.

Рабин-Карп (Rabin-Karp) алгоритм

Түүх: М.О.Рабин, Р.М Карп нар хоёр хэмжээст дүр хайлт гэх мэт бодлогуудыг бодох алгоритмуудыг боловсруулсан байдаг.

Ажиллах зарчим:

1. Тоон дүрслэл: Дүр болон текстийн дэд мөрийг тоо болгон хувиргадаг. (0-9 цифрүүдэд аравтын тооллын систем ашиглана)
2. Хэшлэх(Hash): Дүрын хэш утгыг болон текстийн дэд мөр бүрийн хэш утгыг тооцоолно.
3. Харьцуулах: Хэрэв дүрийн хэш ба текстийн дэд мөрийн хэш ижил байвал тэмдэгт бүрээр шалгана.

Hash: Робин-Карп алгоритмд хэш функц нь тэмдэгт мөрийг тоон утга болгож хувиргадаг.

Хэш утга: $H(S) = (s_1 \cdot d^{m-1} + s_2 \cdot d^{m-2} + \dots + s_{m-1} \cdot d + s_m)$

// d = суурь ихэвчлэн 10

// m = дүрийн урт

Гэхдээ том тоонуудтай ажиллахын тулд модуль (q) ашигладаг.

$H = (H \bmod q)$ ингэснээр хэш утга жижиг удирдахад хялбар болно.

Rolling Hash: Хэрэв эхний хэсэг олдохгүй бол хэш утгыг шинэ хэсэгт шилжүүлэхдээ дахин бүхэлд нь тооцоолохгүй.

- Хуучин хэшээс ахмад тэмдэгтийг хасна.
- Хэшийг d-ээр өсгөж, шинэ тэмдэгтийг нэмнэ.
- Formula:

$$\circ H(p) = (p_1 \cdot d^{m-1} + p_2 \cdot d^{m-2} + \dots + p_{m-1} \cdot d + p_m) \bmod q$$

■ d = суурь (цифр=10, ASCII=256)

■ q = анхны тоо

■ m = дүрийн урт

$$\circ t_{s+1} = (d \cdot (t_s - T[s + 1] \cdot h) + T[s + m + 1]) \bmod q$$

Хугацааны нарийвчлал (Time Complexity):

Дүр хайлтын бодлогын олон хэрэглээнд цөөн тооны, магадгүй тогтмол C байх зөв шилжилтүүдийг тооцоолдог. Ийм хэрэглээнд Робин-Карпын алгоритмын хайлтын дундаж хугацаа нь $O((n - m + 1) + C \cdot m) = O(n + m)$ хугацаа дээр хуурмаг тохирлын шаардагдах хугацааг нэмсэнтэй тэнцүү байна.

- **Хамгийн сайн тохиолдол (Best Case):** $O(n+m)$
 - Хайх загварын хэш ($H(P)$) зөвхөн текстийн зөв байрлалуудад таарна.
 - Хуурмаг таарал огт үүсдэггүй.
 - Урьчилсан бэлтгэл: $O(m)$ - дүр болон эхний дэд мөрийн хэшийг тооцоолох
 - Rolling hash: $O(n)$
- **Хамгийн муу тохиолдол (Worst Case):** $O((n - m + 1) \cdot m)$
 - n = текстийн урт, m = дүрийн урт
 - Бүх дэд мөрүүд ижил хэштэй байх үед алгоритм m тэмдэгтүүдийг бүтэн харьцуулна.
- **Дундаж тохиолдол (Average Case):** $O(n) + O(m(v + n/q))$
 - Хуурмаг тохирлын ховор тохиолдох (Том q , сайн хэш функц)
 - $O(n/q) =$ Хуурмаг тохирлын дундаж тоо

Энд v нь зөв шилжилтүүдийн тоо. Хэрвээ $v = O(1)$ бөгөөд $q \geq m$ байхаар q -г сонгосон бол энэ ажиллах хугацаа нь $O(n)$ байх болно. Өөрөөр хэлбэл хэрвээ зөв шилжилтийн тоо нь дунджаар $O(1)$ буюу зөв шилжилт цөөхөн гардаг бөгөөд анхны тоо q -г дүрийн уртаас их байхаар сонговол алгоритм нь зөвхөн хайлтын $O(n + m)$ хугацаандаа багтаж бүрэн ажиллаж чадна.

Санах ойн нарийвчлал (Space Complexity): $O(1)$

Давуу талууд

- **Олон хэв маяг хайхад тохиромжтой:** Rabin-Karp нь нэг текст дээр олон хэв маягийг нэгэн зэрэг хайх боломжтой (multipattern matching). Хэв маяг бүрт хэш тооцоолж, нэг удаагийн текст гүйлтээр бүгдийг харьцуулж болно.
- **Хурдан дундаж хугацаа:** Дунджаар $O(n+m)$ хугацаатай ажилладаг
- **Хялбар хэрэгжүүлэлт:** Алгоритм нь ойлгоход болон код бичихэд харьцангуй энгийн.

Сул талууд

- **Хамгийн муу тохиолдлын гүйцэтгэл:** Хамгийн муу тохиолдолд $O(n \cdot m)$ хугацаа шаардлагатай

- **Хэш функцээс хамааралтай:**Хэрвээ хэш функц сайн биш бол (жишээ нь, олон тэмдэгт мөр ижил хэш утга руу хувирвал), гүйцэтгэл эрс муудна.
- **Бусад алгоритмаас харьцангуй удаан:**Хамгийн муу тохиолдолд Knuth-Morris-Pratt (KMP) эсвэл Boyer-Moore алгоритмуудтай харьцуулахад Rabin-Karp удаан байдаг.

Тохиромжтой хэрэглээний тохиолдол:

- **Плагиатын илрүүлэлт (Plagiarism Detection):** Баримт бичиг, эссе, эсвэл кодын хэсгүүдээс ижил тэмдэгт мөр эсвэл хэв маягийг хайх.
- **Файлын ялгаа хайх (File Difference Detection):**Хоёр файл эсвэл хувилбар хоорондын ижил хэсгүүдийг олоход (жишээ нь, Git-д ашигладаг diff-ийн ижил төстэй зарчим).

Бойер-Мурын алгоритм

Түүх: Төхтөөс өгөгдсөн тэмдэгт мөрийг хурдан хайх алгоритмыг Роберт Бойер, Жем Мур нар 1977 онд хэрэглэсэн учир алгоритмын ухаанд Бойер-Мурын алгоритм гэж нэрлэгдэх болсон.

Ажиллах зарчим: Энэ алгоритмт нь текстээс өгөгдсөн тэмдэгт мөрийг эхлэлээс нь хайх ба харьцуулалтыг өгөгдсөн тэмдэгт мөрийн төгсгөлөөс эхлэл рүү шилжүүлэн хийдэг. Өгөгдсөн тэмдэгт мөрийн төгсгөлийн тэмдэгтэд хийх харьцуулалтын үр дүнгээс хамаарч дараачийн харьцуулалт эхлэх текстийн байрлалыг тодорхойлдог. Энэ нь өгөгдсөн тэмдэгт мөрийн үсгүүд текстийн үсгүүдтэй хэр давхцаж байгаагаас хамаарч шилжилтийг алгасах алхмыг нэмэгдүүлж хурдан хайх боломжийг бий болгодог.

Тэмдэгт мөр нь 0-255 ASCII кодтой гэж үзээд хайх тэмдэгт мөрийн тэмдгүүдийн кодоор дугаарлалтаа хийсэн pos[256] тоон хүснэгт үүсгэнэ. Энэ хүснэгт нь тухайн тэмдэгт мөрийн эхлэлээс хайж буй тэмдэг ямар дугаарт байгааг илэрхийлнэ. Хэрвээ нэг тэмдэг олон байвал сүүлийн утгаар нь авна. Энэ нь харгалзах жишилт хийж буй тэмдэгтүүд ялгаатай үед дараагийн харьцуулалт эхлэх шилжилтийг тодорхойлно. Иймд хайж буй тэмдэгт мөрийн үсгүүд текстийн үсгүүдтэй хэр давхцаж байгаагаас хамаарч шилжилтийг алгасах алхмыг нэмэгдүүлж хурдан хайх боломжийг бий болгодог.

Heuristic ашиглалт: Boyer-Moore алгоритм нь хоёр төрлийн heuristic (Bad Character болон Good Suffix) ашигладаг тул тэмдэгт мөрийн уртын хэмжээнээс хамааран илүү үр ашигтай гүйлгэх боломжтой болдог.

- **Good Suffix Heuristic:** Тэмдэгт мөрийн нэг хэсэг нь текст дээр таарсан тохиолдолд, текстийг үр ашигтай гүйлгэхийн тулд, өмнөх тохирсон хэсгийг үл тооцох арга
- **Bad Character Heuristic:** Алгоритмын харьцуулах үе шатанд, зөрүү гарсан тэмдэгт нь хайж буй тэмдэгт мөрийн сүүлийн тэмдэгттэй тохирохгүй байвал, текстэн дээр хэрхэн гүйлгэхийг тодорхойлох дүрэм.

Хайлтын чиглэл: Boyer-Moore алгоритм нь текстийг баруун талаас зүүн тийш харьцуулж эхэлдэг бол Naive болон KMP алгоритмууд нь зүүнээс баруун тийш хайлт хийдэг

Хугацааны нарийвчлал (Time Complexity):

- **Хамгийн сайн тохиолдол (Best Case): $O(n/m)$**
 - Хэрэв дүрт байхгүй тэмдэгт текстэнд олон удаа таарвал (жишээ нь, текст="AAAAA", дүр="BCD"), алгоритм хурдан үсрэлт хийнэ.
- **Хамгийн муу тохиолдол (Worst Case): $O(m*n)$**
 - текст ба дүр ижил тэмдэгттэй байх үед, жишээ нь "AAAAAAAAAA" текст ба "AAAA" дүр.
- **Дундаж тохиолдол (Average Case): $O(n)$**
 - Тодорхой бус, гэхдээ санамсаргүй текстэд $O(n)$ байж болно гэж үзэж болно.

Санах ойн нарийвчлал (Space Complexity): $O(m)$

Бэлтгэл шаардлага: m = дүрийн урт

Давуу талууд

- Том текстэнд үр ашигтай, харьцуулалтыг багасгадаг, загварын урт нэмэгдэх тусам илүү хурдан болдог.

Сул талууд

- “Bad Character” болон “Good Suffix” хүснэгтүүдийг урьдчилан бэлтгэх шаардлагатай.

Тохиромжтой хэрэглээний тохиолдол: Энэ алгоритм текст засварлагч, вэб хайлт, мэдээллийн санд загвар хайх, сүлжээний халдлага илрүүлэх системд тохиромжтой.

Хэрэжүүлэлт

Доорх кодыг C програмчлалын хэлээр Visual Studio Code хэрэглэгчийн орчинд бичсэн ба үндсэн оролт гаралтын “stdio” болон тэмдэгт мөртэй ажиллах болон heuristic хүснэгт үүсгэхэд дэмжих зорилгоор “string” сангуудыг ашигласан. Мөн гараас текст уншиж, тэр дотроос дүр үг эсвэл үгийн аль нэг хэсгээр хайж, хайлтын үр дүнг индексээр болон текстийн хэд дэх үг болохийг буцаах команд бүхий кодыг бичлээ.

Бойер-Мурын алгоритм

```
C BoyerMoore.c > countWords(char *, int)
1  #include <stdio.h>
2  #include <stdbool.h>
3  #include <string.h>
4
5  #define NO_OF_CHARS 256
6
7  int max(int a, int b) { return (a > b) ? a : b; } // hoyr toonii ihiin oloh
8
9  // bad char heuristic husnegt uusgeh
10 void badCharHeuristic(char* str, int size, int badchar[NO_OF_CHARS]) {
11     for (int i = 0; i < NO_OF_CHARS; i++)
12         badchar[i] = -1; // buh temdegtuudiig -1 utgatai bolgoh
13
14     for (int i = 0; i < size; i++) // temdegt muriig suuliin temdegtuudiig indexeer ni temdegleh
15         badchar[(int)str[i]] = i;
16 }
17
18 int countWords(char* text, int index) { // ugiin too tootsooloh
19     int count = 0;
20     int inWord = false;
21     int length = strlen(text);
22     if (index > length) index = length;
23     for (int i = 0; i < index; i++)
24         if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t') {
25             inWord = false;
26         } else if (!inWord) {
27             count++;
28             inWord = true;
29         }
30     return count;
31 }
32
33 void search(char* txt, char* pat) { //boyer moore algorithmiig heregjuuleh
34     int m = strlen(pat);
35     int n = strlen(txt);
36
37     int badchar[NO_OF_CHARS]; // badchar husneat uusgeh
38
39     int s = 0; // text deer guigeh index
40     int foundCount = 0; // duriig tooloh
41     while (s <= (n - m)) {
42         int j = m - 1;
43
44         while (j >= 0 && pat[j] == txt[s + j]) // temdegt muriig tugsguluus ni ehlej shalgah
45             j--;
46
47         if (j < 0) {
48             int wordIndex = countWords(txt, s) + 1; // oldson ugiin too
49             printf("\nOruulsan ugiin ehni index: %d tus ugiin texted ezleh bair: %d", s, wordIndex);
50             // herev buh temdegt taarsan bol hevleh
51             foundCount++;
52
53             // Daraagin bairlal ruu shiljih
54             s += (s + m < n) ? m - badchar[txt[s + m]] : 1;
55         } else {
56             s += max(1, j - badchar[txt[s + j]]); // bad char-iin durmiig ashiglaaj guilgeh hemjeeg tootsooloh
57         }
58     }
59
60     printf("\n\nNiit %d удаа oldloo.\n", foundCount);
61 }
62
63 int main() {
64     char txt[1000], pat[1000];
65
66     printf("Text oruulna uu: ");
67     fgets(txt, sizeof(txt), stdin);
68     txt[strcspn(txt, "\n")] = '\0'; // shine muriig ustgah
69
70     printf("Haih ded muriig oruulna uu: ");
71     fgets(pat, sizeof(pat), stdin);
72     pat[strcspn(pat, "\n")] = '\0';
73     printf("\nText: \"%s\"", txt);
74     printf("Haih ded mur: \"%s\"", pat);
75
76     search(txt, pat);
77     return 0;
78 }
79
80
81 }
```

Жишээ үр дүн

```
Text oruulna uu: bad character heuristic and the good suffix heuristic. When a mismatch occurs, the bad character heuristic determines the shift distance based on the last occurrence of the mismatched character in the pattern
Haih ded muriig oruulna uu: the

Text: "bad character heuristic and the good suffix heuristic. When a mismatch occurs, the bad character heuristic determines the shift distance based on the last occurrence of the mismatched character in the pattern"
Haih ded mur: "the"

Oruulsan ugiin ehnii index: 28 tus ugiin texted ezleh bair: 5
Oruulsan ugiin ehnii index: 79 tus ugiin texted ezleh bair: 13
Oruulsan ugiin ehnii index: 118 tus ugiin texted ezleh bair: 18
Oruulsan ugiin ehnii index: 146 tus ugiin texted ezleh bair: 23
Oruulsan ugiin ehnii index: 169 tus ugiin texted ezleh bair: 27
Oruulsan ugiin ehnii index: 197 tus ugiin texted ezleh bair: 31

Niit 6 удаа oldloo.
[1] + Done "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microsoft-MIEngine-In-j1tajdk2.ihh" 1>
```

(текстээс "the" дүрээр хайсан эхний хайлтын үр дүн)

```
Text oruulna uu: asd asdfasdf asdf asdfa adfasdf
Haih ded muriig oruulna uu: b

Text: "asd asdfasdf asdf asdfa adfasdf"
Haih ded mur: "b"

Niit 0 удаа oldloo.
[1] + Done "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microsoft-MIEngine-In-l2ikxbhq.do3" 1>
```

(текстэд байхгүй тэмдэгт хайхад 0 олдлоо гэж хэвлэнэ)

Рабин-Карп (Rabin-Karp) алгоритм

```
C BoyerMoore.c C RabinKarp.c x
C RabinKarp.c > search(char[], char[], int)
4 void search(char pat[], char txt[], int q) // rabin karp herejuuleh algorithm
5 {
6     int M = strlen(pat);
7     int N = strlen(txt);
8     int i, j;
9     int p = 0; // duriin hash utga
10    int t = 0; // textiin hash utga
11    int h = 1;
12    int d = 256; // ascii 0-255
13    int found = 0;
14    for (i = 0; i < M - 1; i++) // h iin utga "pow(d, M-1)%q"
15        h = (h * d) % q;
16
17    for (i = 0; i < M; i++) { // dur болон ehnii hash utgiig tootsooloh
18        p = (d * p + pat[i]) % q;
19        t = (d * t + txt[i]) % q;
20    }
21
22    for (i = 0; i <= N - M; i++) {
23        if (p == t) {
24            for (j = 0; j < M; j++) {
25                if (txt[i + j] != pat[j])
26                    break;
27            }
28            if (j == M)
29                printf("\nOruulsan ugiin ehnii index: %d", i);
30            found = 1;
31        }
32        if (i < N - M) {
33            t = (d * (t - txt[i] * h) + txt[i + M]) % q;
34            if (t < 0)
35                t = (t + q);
36        }
37    }
38    if (!found) {
39        printf("dur textnd baihui\n");
40    }
41
42 int main() {
43     char txt[1000], pat[1000];
44
45     printf("Text oruulna uu: ");
46     fgets(txt, sizeof(txt), stdin);
47     txt[strlen(txt)] = '\0';
48
49     printf("Haih ded muriig oruulna uu: ");
50     fgets(pat, sizeof(pat), stdin);
51     pat[strlen(pat)] = '\0';
52
53     printf("\nText: \"%s\"", txt);
54     printf("Haih ded mur: \"%s\"", pat);
55
56     int q = 101;
57     search(pat, txt, q);
58     return 0;
59 }
```


Жишээ үр дүн

```
Text oruulna uu: bad character heuristic and the good suffix heuristic. When a mismatch occurs, the bad character heuristic determines the shift distance based on the last occurrence of the mismatched character in the pattern
Haih ded muriig oruulna uu: the

Text: "bad character heuristic and the good suffix heuristic. When a mismatch occurs, the bad character heuristic determines the shift distance based on the last occurrence of the mismatched character in the pattern"
Haih ded mur: "the"

Oruulsan ugiin ehnii index: 28
Oruulsan ugiin ehnii index: 79
Oruulsan ugiin ehnii index: 118
Oruulsan ugiin ehnii index: 146
Oruulsan ugiin ehnii index: 169
Oruulsan ugiin ehnii index: 197[1] + Done
/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microso
```

(текстээс "the" дүрээр хайсан эхний хайлтын үр дүн)

```
Text oruulna uu: 987654321098765432109876543210
Haih ded muriig oruulna uu: 54321

Text: "987654321098765432109876543210"
Haih ded mur: "54321"

Oruulsan ugiin ehnii index: 4

Oruulsan ugiin ehnii index: 14

Oruulsan ugiin ehnii index: 24
[1] + Done
/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/Microsoft-MIEngine-In-svpans0t.q5k" 1
```

(текстээс "54321" дүрээр хайсан эхний хайлтын үр дүн)

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Text oruulna uu: 123456789
Haih ded muriig oruulna uu: 0

Text: "123456789"
Haih ded mur: "0"
dur textnd baihgui
```

(текстээс "0" буюу байхгүй тэмдэгтийг хайхад" хайсан эхний хайлтын үр дүн)

Харьцуулалт

Features	Time complexity(Best)	Time complexity(Worst)	Time complexity(Average)	Space complexity	Preprocessing
Boyer-Moore	$O(n/m)$	$O(m*n)$	$O(n)$	$O(m)$	$O(m + k)$
Rabin-Karp	$O(n+m)$	$O((n - m + 1)* m)$	$O(n+m)$	$O(1)$	$O(m)$

Linux-ийн time командаар 2 алгоритмыг харьцуулав.

Time command ажилласны дараа 3 хугацаа харуулна.

- Real (бодит хугацаа) -
- User (хэрэглэгчийн хугацаа) - CPU-д програмын ажиллахад зарцуулсан хугацаа
- Sys (системийн хугацаа) - CPU-д системийн дуудлага (kernel) ажиллахад зарцуулсан хугацаа

User хугацаа нь хамгийн чухал. Алгоритмын цэвэр цэвэр гүйцэтгэлийн хэмжүүр юм.

```
munkhpurev@munkhpurev-HP-Laptop-14s-dq5xxx:~/datastructure$ gcc RabinKarp.c -o RabinKarp
munkhpurev@munkhpurev-HP-Laptop-14s-dq5xxx:~/datastructure$ time ./RabinKarp
Text oruulna uu: bad character heuristic helps determine how far to shift the pattern to the right when a mismatch occurs, based
on the last occurrence of the mismatched character in the pattern
Haish ded muriig oruulna uu: the

Text: "bad character heuristic helps determine how far to shift the pattern to the right when a mismatch occurs, based on the la
st occurrence of the mismatched character in the pattern"
Haish ded mur: "the"

Oruulsan ugiin ehni index: 57
Oruulsan ugiin ehni index: 72
Oruulsan ugiin ehni index: 115
Oruulsan ugiin ehni index: 138
Oruulsan ugiin ehni index: 166
real    0m5.656s
user    0m0.003s
sys     0m0.004s
```

```

munkhpurev@munkhpurev-HP-Laptop-14s-dq5xxx:~/datastructure$ gcc RabinKarp.c -o RabinKarp
munkhpurev@munkhpurev-HP-Laptop-14s-dq5xxx:~/datastructure$ time ./RabinKarp
Text oruulna uu: 123456789
Haih ded muriig oruulna uu: 0

Text: "123456789"
Haih ded mur: "0"
dur textnd baihgui

real    0m5.127s
user    0m0.003s
sys     0m0.005s
munkhpurev@munkhpurev-HP-Laptop-14s-dq5xxx:~/datastructure$

```

(RabinKarp algortihm Linux-time command)

```

munkhpurev@munkhpurev-HP-Laptop-14s-dq5xxx:~/datastructure$ gcc BoyerMoore.c -o BoyerMoore
munkhpurev@munkhpurev-HP-Laptop-14s-dq5xxx:~/datastructure$ time ./BoyerMoore
Text oruulna uu: bad character heuristic helps determine how far to shift the pattern to the right when a mismatch occurs, based
on the last occurrence of the mismatched character in the pattern
Haih ded muriig oruulna uu: the

Text: "bad character heuristic helps determine how far to shift the pattern to the right when a mismatch occurs, based on the la
st occurrence of the mismatched character in the pattern"
Haih ded mur: "the"

Oruulsan ugiin ehnii index: 57 tus ugiin texted ezleh bair: 10
Oruulsan ugiin ehnii index: 72 tus ugiin texted ezleh bair: 13
Oruulsan ugiin ehnii index: 115 tus ugiin texted ezleh bair: 21
Oruulsan ugiin ehnii index: 138 tus ugiin texted ezleh bair: 25
Oruulsan ugiin ehnii index: 166 tus ugiin texted ezleh bair: 29

Niit 5 udaa oldloo.

real    0m4.942s
user    0m0.002s
sys     0m0.003s

```

```

munkhpurev@munkhpurev-HP-Laptop-14s-dq5xxx:~/datastructure$ gcc BoyerMoore.c -o BoyerMoore
munkhpurev@munkhpurev-HP-Laptop-14s-dq5xxx:~/datastructure$ time ./BoyerMoore
Text oruulna uu: 123456789
Haih ded muriig oruulna uu: 0

Text: "123456789"
Haih ded mur: "0"

Niit 0 udaa oldloo.

real    0m9.418s
user    0m0.002s
sys     0m0.003s

```

(BoyerMoore algortihm Linux-time command)

	Time complexity (Average)	Time complexity (command)
<i>BoyerMoore algortihm</i>	$O(n)$	$O()= 0.002c$
<i>RabinKarp algortihm</i>	$O(n+m)$	$O()= 0.003c$

Дүгнэлт

Naive, Rabin-Karp, Boyer-Moore гэсэн гурван мөр хайлтын алгоритмыг авч үзэхэд тус бүр өөрийн давуу болон сул талуудтай бөгөөд хэрэглээний онцлогоос хамааран тохиромжтой байдал нь өөрчлөгддөг.

- **Naive алгоритм** нь энгийн бөгөөд ойлгоход хялбар тул жижиг хэмжээний текст эсвэл прототип хөгжүүлэлтэд тохиромжтой. Гэвч хугацааны нарийвчлал ($O((n - m + 1) * m)$) өндөртэй тул том өгөгдөлд үр ашиггүй, удаан ажилладаг. Санах ойн хэрэглээ бага ($O(1)$) боловч практикт хязгаарлагдмал хэрэглээтэй.
- **Rabin-Karp алгоритм** нь хэш функцийг ашиглан дундаж тохиолдолд $O(n + m)$ хугацаатай ажилладаг бөгөөд олон загвар хайхад тохиромжтой. Гэсэн хэдий ч хамгийн муу тохиолдолд ($O((n - m + 1) * m)$) гүйцэтгэл мууддаг ба хэш функцээс ихээхэн хамааралтай. Санах ойн хэрэглээ $O(1)$ тул хөнгөн боловч бусад алгоритмуудтай харьцуулахад хамгийн муу тохиолдолд удаан байж болно.
- **Boyer-Moore алгоритм** нь Bad Character болон Good Suffix heuristic-ийг ашиглан том текстэнд харьцуулалтыг багасгаж, хамгийн сайн тохиолдолд $O(n/m)$, дундаж тохиолдолд $O(n)$ хугацаатай ажилладаг тул текст засварлагч, мэдээллийн сангийн хайлт зэрэгт өндөр үр ашигтай. Гэхдээ урьдчилсан бэлтгэл шаардлагатай ($O(m + k)$) бөгөөд санах ойн хэрэглээ $O(m)$ нь Naive болон Rabin-Karp-тай харьцуулахад өндөр.

Linux-ийн time командаар харьцуулахад Boyer-Moore (0.002 сек) нь Rabin-Karp (0.003 сек)-аас дунджаар хурдан ажилласан нь түүний үр ашгийг харуулж байна. Гэсэн ч Rabin-Karp олон загвар хайхад давуу талтай бол Boyer-Moore нь ганц загварыг хурдан хайхад илүү тохиромжтой.

Ерөнхий дүгнэлт: Хэрэв жижиг, энгийн хайлт хийх бол Naive, олон загвар хайх бол Rabin-Karp, том текстэнд хурдан хайлт хийх бол Boyer-Moore алгоритмыг сонгох нь зүйтэй. Практикт Boyer-Moore нь дундаж гүйцэтгэлээрээ тэргүүлдэг ч хэрэглээний онцлог, өгөгдлийн хэмжээ зэргээс хамаарч алгоритмыг сонгох хэрэгтэй.

Эх сурвалж

- Introduction to Algorithms, 3rd edition MIT Press
- Алгоритм ба програмчлал Д.Гармаа
- Алгоритмчлол ба програмчлалын үндэс Д. Гармаа
- [Geeksforgeeks.org](https://www.geeksforgeeks.org/)